

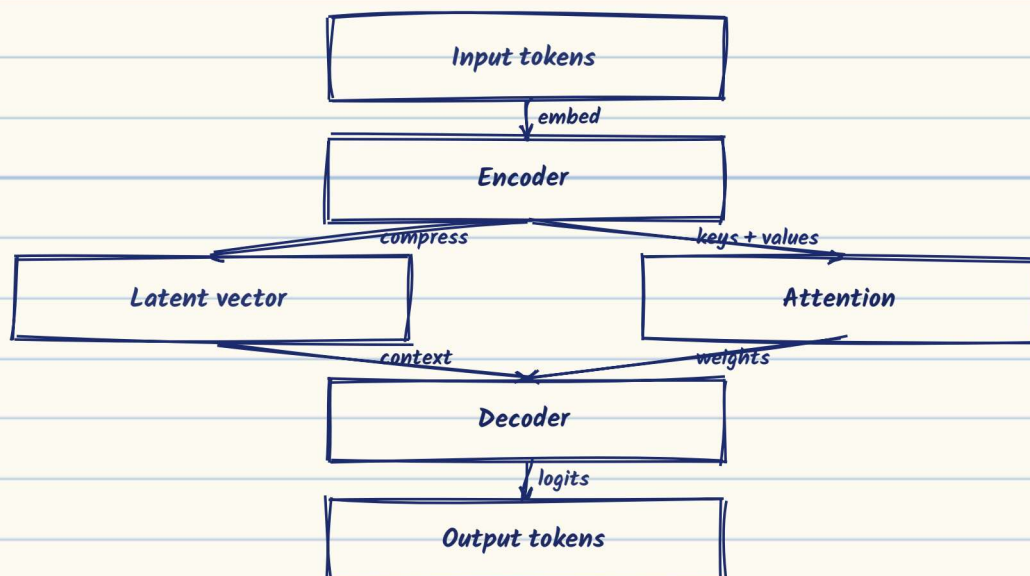
Encoder-Decoder Models

An encoder-decoder model splits the problem in two. The encoder reads the whole input sequence and compresses what it understood into a fixed representation; the decoder reads that representation and writes the output sequence one token at a time. Nothing forces the two halves to share an architecture, which is why the pattern outlived the recurrent networks it was invented for.

Latent vector — The fixed-size representation the encoder hands to the decoder — everything the model managed to keep about the input.

e.g. A 512-dimensional vector standing in for a whole sentence.

How the halves connect



ATTENTION

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

IN SHORT

1. The encoder compresses, the decoder generates
2. Attention lets the decoder look back at every input position
3. The two halves need not share an architecture

Training It

Teacher forcing vs free running

Teacher forcing	Free running
Decoder sees the true previous token	Decoder sees its own last prediction
Trains fast, converges early	Trains slowly, often unstably
Can be fully parallelised over time	Strictly sequential
Mismatched with inference	Matches inference exactly
Suffers exposure bias	No exposure bias
Standard for pre-training	Used for fine-tuning and RL
Loss is optimistic	Loss reflects real generation
Cheap to implement	Needs a sampling loop

```
python
import torch
import torch.nn as nn

class Decoder(nn.Module):
    def __init__(self, vocab, hidden):
        super().__init__()
        self.embed = nn.Embedding(vocab, hidden)
        self.rnn = nn.GRU(hidden, hidden, batch_first=True)
        self.out = nn.Linear(hidden, vocab)

    def forward(self, tokens, state):
        # tokens: (batch, time) of the PREVIOUS true tokens.
        x = self.embed(tokens)
        h, state = self.rnn(x, state)
        return self.out(h), state

def train_step(model, batch, opt, pad_id):
    loss_fn = nn.CrossEntropyLoss(ignore_index=pad_id)
    logits, _ = model(batch.inputs, batch.state)
    loss = loss_fn(
        logits.reshape(-1, logits.size(-1)),
```

```
python (contd.)  
    batch.targets.reshape(-1),  
    )  
    opt.zero_grad()  
    loss.backward()  
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)  
    opt.step()  
    return loss.item()
```

★ **Clip the gradients. Recurrent decoders explode without it, and the failure looks like a bad learning rate for a long time before you find it.**

e.g. Why exposure bias matters in practice

A model trained only with teacher forcing has never once seen its own mistake as input. The first wrong token at inference puts it in a state it was never trained on, and the rest of the sentence drifts.

IN SHORT

- 1. Teacher forcing is fast but optimistic*
- 2. Clip gradients or the decoder explodes*
- 3. Exposure bias is the price of the fast path*

How We Got Here

Sequence models, roughly

1997

LSTM — gates make long-range gradients survive.

2014

Sequence-to-sequence learning; the encoder-decoder split.

2015

Attention, so the decoder stops relying on one vector.

2017

Transformers drop recurrence entirely.

2018

Pre-train then fine-tune becomes the default recipe.

2020

Scaling laws turn architecture choices into budget choices.

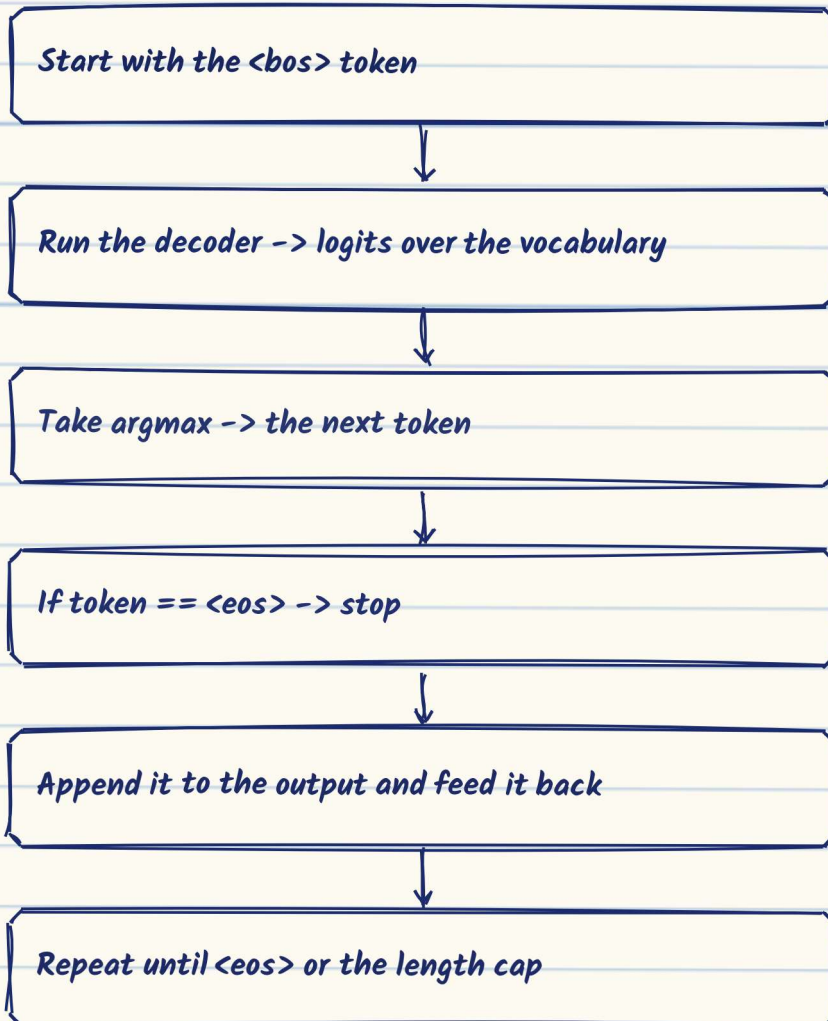


IN SHORT

1. Every fix is about shortening the path a gradient travels
2. Attention won because it parallelises
3. Recurrence was the constraint, not the goal

Greedy Decoding

One decoding step



Why greedy is not enough

- A locally best token can strand the rest of the sentence
- No way to recover from an early mistake
- Beam search keeps k hypotheses instead of one
- Sampling trades accuracy for variety

Beam search is the usual answer, and it is worth remembering that it is still an approximation — the true argmax over whole sequences is intractable, so every decoder in production is making a guess about which partial guesses to keep.

IN SHORT

1. Greedy takes the best token, not the best sentence
2. Beam search keeps k candidates alive
3. Both are approximations of an intractable search